School of Future Tech

Case Study Report

on

RBAC System* (Cpp*)

by

Name : *Asad Antule*

Roll number : *150096725138*

Index

1. *Introduction to the Case Study.*
2. *Problem Statement / Case Background (Abstract).*
3. *Problem Statement / Case Study Design.*
4. *Methods & Algorithms Technology Applied in the Problem Statement / Case Study.*
5. *Problem Statement / Case Study Implementation Details and Snapshots.*
6. *Problem Statement / Case Study Results and Conclusion.*
7. *References*

Introduction to the Case Study

Role-Based Access Control (RBAC) is a foundational security protocol used to manage user permissions within an organization. Instead of assigning permissions to every individual user, access is granted based on defined "Roles." This project utilizes C++ to simulate an environment where system functionalities like Reading, Writing, and Deleting data are restricted based on whether a user is an Admin, Manager, or Employee.

Problem Statement / Case Background (Abstract)

In any multi-user environment, unauthorized access to sensitive data is a critical threat to system security. Giving every user full control over data can lead to accidental or malicious data loss. This case study addresses the need for a programmatic "gatekeeper" that identifies a user's organizational rank and strictly enforces the security policies associated with that rank.

Problem Statement / Case Study Design

The system is designed using **Object-Oriented Programming (OOP)**. The design focuses on three pillars:

- **Identity Management:** Storing user credentials.
- **Persistent Storage:** Using flat files to ensure data is saved permanently.
- **Authorization Logic:** A switch-case and conditional hierarchy that maps roles to specific permissions.

Methods & Algorithms / Technology Applied

The project is built using the following technologies and methodologies:

- **C++ Language:** The core programming language.
- **File Handling (<fstream>):** Used ofstream to create/append users and ifstream to read and verify roles from users.txt.
- **Encapsulation:** The User class is used to group user attributes.
- **Search Algorithm:** A linear search is performed within the text file to find and match the username to its corresponding role.

Implementation Details and Snapshots

The implementation is modular, ensuring the code is easy to debug and maintain.

Key Logic Segments:

- **createUser()**: Opens users.txt to add new users without deleting old ones.
- **getRole()**: Parses the text file and returns the string value of the role (Admin/Manager/Employee).
- **checkAccess()**: The main security function that compares the retrieved role against the user's menu choice.

Snapshots:

- **Snapshot 1 (Registration):**

```
===== RBAC SYSTEM =====
1. Create User
2. View Users
3. Login & Check Access
4. View Role Permissions
5. Exit
Enter Choice: 1
Enter Username: Asad
Enter Role (Admin/Manager/Employee): Admin
User Created Successfully!
```

- **Snapshot 2 (File Structure):**

CPP > users.txt

```
Asad Admin
User Employee
```

- **Snapshot 3 (Access Control):**

```
===== RBAC SYSTEM =====
1. Create User
2. View Users
3. Login & Check Access
4. View Role Permissions
5. Exit
Enter Choice: 3
Enter Username: User
Logged in as: Employee

Select Operation
1. Read Data
2. Write Data
3. Delete Data
Enter Choice: 3
Access Denied
```

Results and Conclusion

Results:

The system successfully differentiates between user roles. During testing, an "Admin" was able to access all operations, while an "Employee" was restricted to "Read Only" access. Data remained intact across multiple program restarts due to successful file handling implementation.

Conclusion:

This project effectively satisfies the requirements of **Case Study 57**. It demonstrates that C++ can be used to build secure, role-aware applications. The implementation of file-based storage and conditional permission logic provides a robust foundation for more advanced Cyber Security projects in the future.

References

1. **ITM Skills University**: *Semester II Course Material on Object-Oriented Programming*.
2. **Assignment Sheet**: *Case Study 57: Role-Based Access Control (RBAC) System*.
3. **Bjarne Stroustrup**: *The C++ Programming Language (4th Edition)*.
4. **Cplusplus.com**: *Documentation for fstream and file I/O operations*.